A Comparison between Large Language Models and Traditional Predictive Models in a Complex Classification Task

Thien-Lac Quach[1,2][0009−0004−6310−5149] and Ton-Minh-Ky Tran[1,2][0009−0006−5027−2516]

[1] Faculty of Information Technology, University of Science, Ho Chi Minh City, Vietnam
[2] Vietnam National University, Ho Chi Minh City, Vietnam
{24125092,24125102}@apcs.fitus.edu.vn

Abstract. In this paper, we compare the performance of Large Language Models (LLMs) and traditional machine learning models on a complex classification task in education: predicting students' final academic outcomes. We evaluate two LLMs prompted using different techniques, benchmarked against a variety of traditional machine learning models. Our results indicate that despite their complexity and size, LLMs heavily underperform compared to traditional models in this specific task. These findings suggest that general-purpose LLMs, whose main strength is in natural language processing, are not as suitable for structured classification tasks as traditional counterparts. In our future work, we aim to explore and further validate the reasons behind this discrepancy, and investigate potential improvements in prompting techniques or model architectures to enhance the performance of LLMs-based approaches in classification tasks.

Keywords: Large Language Models · Traditional Machine Learning Models · Classification problem · Prompting techniques · Educational context

1 Introduction

The prevalence of Large Language Models (LLMs) in education has opened up new avenues for enhancing student learning and academic support. LLMs have the potential to analyze vast amounts of educational data, identify patterns, and provide personalized recommendations to students and educators alike. In this study, we focus on leveraging the capabilities of LLMs to analyze students examination results and identify the level of assistance they require to succeed academically.

Using multiple prompting techniques, we analyze a given dataset of students' examination results, split into training and testing sets, to identify patterns and factors that contribute to academic success or failure in order to provide an appropriate level of academic support. We classify students' level of assistance required into different categories based on their performance, then instruct the LLMs to train themselves with the training sets and evaluate their performance on the testing sets.

By identifying at-risk students early on, Higher Education Institutions (HEIs) can implement strategies to improve retention rates and enhance the overall educational experience; students can also benefit from the predictions provided by the LLMs as an early-warning system. The results of this study can inform policy decisions and contribute to the development of more effective support systems for students in HEIs.

In order to determine the scope of our work, we begin by utilizing the systematic literature review framework devised by [21]. To this end, section 2 and 3 will elaborate on models background and choice, data collection, and evaluation metrics. Section 4 and 5 will describe present the methodology and experimental setup, including data preprocessing, feature selection, and model training. Finally, Section 6 will discuss the results and implications of our findings, as well as potential future research directions in this area.

2 Background

2.1 Traditional Machine Learning Models

This paper chooses six traditional machine learning models for the classification task. These include: Random Forest, Support Vector Machine, Gradient Boosting, K-Nearest Neighbors, Extreme Learning Machines (ELM), and Multilayer Perceptron (MLP). The reason for selecting these models will be explained in the Methodology section. Below is a brief introduction to each of these models.

Random Forest. Random Forest (RF) is a ensemble learning method first proposed by Ho in 1995 [17] then formally developed by Breiman and Cutler in 2006 [4]. The algorithm generates ("grow") multiple decision trees to vote for the final prediction, whose growth is each decided by a random vector Θ, made independently from past vectors though with the same distribution. The final prediction of the Random Forest is made by aggregating the predictions, also called "voting", of all individual trees, typically through majority voting for classification tasks [4]. RF is known for its robustness to overfitting, ability to han-

dle high-dimensional data, and effectiveness in capturing complex interactions between features [4]. It is widely used in various applications, including classification and regression tasks [4] and remains one of the most effective machine learning models, despite its early introduction [14].

Support Vector Machine. Support Vector Machine (SVM) is a supervised learning model that implement a simple idea: to find the optimal hyperplane that best separates the input data in a feature space into different classes. The optimal hyperplane is defined to be the one that maximizes the margin between the two classes, which is the distance between the hyperplane and the nearest data points from each class taken from training data, known as support vectors [9]. SVM can be extended to handle non-linearly separable data by using kernel functions, particularly Gaussian Radial Basis Function (RBF) kernel, which maps the input data into a higher-dimensional feature space where a linear separation is possible [25]. SVM is widely used for classification tasks due to its effectiveness in high-dimensional spaces and its ability to model complex relationships between features and classes [18].

Gradient Boosting. Gradient Boosting (GB) is an ensemble learning method that builds a series of weak learners, typically trees, that learns from the residuals of previous learners. Residuals are defined to be the differences between the predicted values and the actual values of the target variable.[8].The training algorithm of GB iterates through M boosting rounds, where in each round m, a new weak learner $T_{(m)}$ is trained to predict the residuals, thereby rectifying mistakes made by the previous learners.[8] The final prediction of the GB model is obtained by summing the predictions of all weak learners, weighted by a learning rate ν that controls the contribution of each learner to the final model [15]. The prediction process work as follows: for each tree $T_{(m)}$ and test instance x_i, the output $T_m(x_i)$ computed then scaled by learning rate ν and added to the cumulative prediction from previous trees, resulting in the final prediction

$$F_M(x_i) = \sum_{m=1}^{M} \nu T_m(x_i) \qquad ([8])$$

GB is known for its high predictive accuracy and is widely used in various applications, including classification and regression tasks [15].

K-Nearest Neighbors. K-Nearest Neighbors (KNN) is a non-parametric, instance-based learning algorithm that classifies a data point based on the majority class among its k nearest neighbors in the feature space [10]. Distance between data points is typically measured by Euclidean distance, although other measures such as Pearson or Spearman distance can also be used [3]. The choice of the algorithm's sole hyperparameter, k, is crucial for the performance of KNN. It is observed that a large value of k can make the model be more independent of noise, though reducing interclass boundaries.[13]. Due to its simplicity, KNN's accuracy can be significantly affected by the choice of k and the nature of the data, often require feature scaling to ensure that all features contribute equally to the distance calculations [23]. KNN is noted

for its strong consistency: as the size of data increases, the probability of KNN making an error converges to no worse than twice the Bayes error rate, which is the lowest possible error rate for any classifier [10].

Extreme Learning Machines. Extreme Learning Machine (ELM) is a type of feedforward neural network, first proposed by Huang et al. in 2006 [20]. ELM is meant to be a learning algorithm for single-hidden layer feedforward neural networks (SLFNs) whose speed is significantly faster than traditional counterparts. ELM randomly assigns the input weights and hidden layer biases, and then analytically determines the output weights by solving a linear system of equations [20]. The training process of ELM consists of three steps: (1) randomly generate the input weights and hidden layer biases; (2) compute the hidden layer output matrix; (3) calculate the output weights by using the Moore-Penrose generalized inverse of the hidden layer output matrix [20]. ELM has been shown to achieve good generalization performance with significantly reduced training time compared to traditional feedforward neural networks, making it suitable for large-scale classification tasks, being able to surpass the performance of SVM in some regression and classification tasks [19].

Multilayer Perceptron. Multilayer Perceptron (MLP) is a class of feedforward artificial neural network that consists of at least three layers: an input layer, one or more hidden layers, and an output layer [24]. First proposed by Rosenblatt in 1958, MLP also serves as a basis for more complex neural network architectures, upon which ELM is built. Crucial in MLP architecture are the concepts of units and hidden layers. Units, also known as neurons, are the basic computational elements of MLP that receive input, process it, and produce output. Hidden layers are the layers of units that exist between the input and output layers, allowing MLP to learn complex representations of data [24]. MLP functions by assigning weights to the connections between units and applying an activation function (commonly a sigmoid or ReLU function) to the output of each neuron. Learning in MLP is typically achieved through backpropagation and gradient descent algorithms, which adjust the weights to minimize the error between the predicted and actual outputs (modeled by a loss function) [16].

2.2 Large Language Models.

Large Language Models (LLMs) presents a significant advancement in artificial intelligence, particularly in the field of natural language processing (NLP). The architecture of LLMs is based on the Transformer model, first introduced in a foundational paper by Vaswani et al. in 2017 [27]. The Transformer architecture utitlizes self-attention mechanisms to process input data, allowing it to capture long-range dependencies and contextual information effectively [27]. LLMs are trained on an enormous amount of data and often possess billions of parameters, enabling them to generate human-like text and perform a wide range of language tasks with high accuracy [5]. The training process of LLMs typically involves two stages: pre-training and fine-tuning. During pre-training, the model learns general language patterns

from a large corpus of text data, while fine-tuning involves further training the model on specific tasks or domains to enhance its performance in those areas [11]. The existence of LLMs revolutionized the field of artificial intelligence, in that they are the first models that can generalize across a wide range of tasks without task-specific training, and they have been shown to outperform traditional machine learning models in various NLP tasks [5].

For our paper, we choose two models: **Qwen-2.5-3B** and **LLaMa-2-8B** (the suffix XB of each model's name refers to the number of billions of parameters X they have). These models are publicly available on the platform Hugging Face. Below is a brief introduction to each of these models.

Qwen-2.5-3B. Qwen-2.5-3B is a large language model developed by Alibaba, which is part of the Qwen series of models. The Qwen 2.5 series is deesigned to be more efficient than its predecessors, especially on coding and mathematical tasks. It has also been reported to enhance instruction-following, producing longer responses (beyond 8K tokens), interpreting structured inputs (e.g., tables), and generating structured outputs, particularly in JSON format. It is also more robust to variations in system prompts, which improves role-playing behavior and condition control in chatbot applications.

LLaMa-3-8B. LLama-3-8B is a large language model developed by Meta, which is part of the LLaMa series of models. It is designed to be more efficient and capable across diverse language tasks. Architecturally, LLaMa 3.1 is an autoregressive language model built on an optimized Transformer backbone. Its tuned variants are aligned with human preferences for helpfulness and safety through supervised fine-tuning (SFT), followed by reinforcement learning from human feedback (RLHF).

3 Related Work

3.1 Machine Learning Models as Classifiers in Educational context

3.2 Data aggregation

It stands to reason that gathering high-quality and representative data is often a difficult endeavour. [2] delineated a framework where a list of hypothetically relevant features are enumerated and filtered via information gain. More specifically, the features that are taken into consideration include those relating to students' study hours, participation in extracurricular activities, hours of sleep, and mock exams practiced [26]. Surveys for these features are deployed online as a questionaire via the platform Google Forms.

3.3 Performance metrics

Common metrics to assess multimodal classification algorithm include accuracy, precision, recall, F1-score [2, 29] and area under the receiver operating characteristic curve (AUC-ROC) [6, 28]. These metrics provide insights into the model's ability to correctly classify instances, handle imbalanced datasets, and make informed decisions based on the predictions [6]. Techniques for handling imbalanced datasets are crucial when working with DT-based models to avoid bias towards the majority class

in the models. Here, we focus on Synthetic Minority Over-sampling Technique (SMOTE), specifically its variant ADASYN, which utilies a weighted distribution to determine the learning difficulty of minority class samples and generates synthetic data accordingly [7, 12]. This approach helps to balance the dataset and improve the performance of the decision tree models in predicting the minority class.

4 Methodology

In this section, we explain the methodology used to conduct our research. We first describe the dataset and preprocessing pipeline. Next, we outline the traditional machine learning models and LLMs we employed for our experiments. Finally, we discuss the evaluation metrics and experimental setup.

4.1 Dataset and Preprocessing

We utilized a dataset named Open University Learning Analytics dataset (hereby referred to as OULAD), described in [22]. This dataset reports the background, behavior, and performance of students from England in a given Virtual Learning Environment (VLE). The dataset contains 32,593 records of 20 features, including demographic information, course interactions, and final results. The features include both categorical and numerical data, which required preprocessing before model training. The dataset, as outlined by its authors, consists of four modules:

Student Demographics: This module includes background information about the students, such as age, gender, and educational as well as socio-economic background. One notable feature is the Index of multiple deprivation (IMD), which is a measure of socio-economic status based on the students' postal codes.

Course Registrations: This module contains information about the courses students are registered for, including course/presentation codes and their lengths. It also includes data students' registration and unregistration dates.

Assessment Results: This module captures the students' performance in three types of assessments, namely Tutor Marked Assessments (TMA), Computer Marked Assessments (CMA), and Final Exams (Exam), adjusted by weights. It provides insights into their academic progress and outcomes.

VLE interactions: This module records the students' interactions with the Virtual Learning Environment (VLE), including the number of clicks on various pages and resources. It also includes the dates of these interactions, which can be used to analyze engagement patterns over time. This module also provides information on the resources in the VLE.

Module	**Table**	**Records**	**What it contains (main fields)**
Student Demographics	`studentInfo`	32,593	Student profile and outcome labels: `id_student`, demographics (gender, region, education, disability), socio-economic and age bands, prior attempts, studied credits, and `final_result`.
Course Registrations	`courses`	22	Module-presentation metadata: `code_module`, `code_presentation`, and course `length`.
Registrations	`studentRegistration`	32,593	Enrollment timeline per student and presentation: `id_student`, `date_registration`, and `date_unregistration`.

Table 1: OULAD: Student and Course Demographics

Module	**Table**	**Records**	**What it contains (main fields)**
Assessment Results	assessments	206	Assessment structure: `code_module`, `code_presentation`, `id_student`, `id_assessment`, `assessment_type`, `date`, `weight`
Assessment Results	studentAssessment	173,912	Student performance: `id_student`, `id_assessment`, `is_banked`, `date_submitted`, `score`

Table 2: OULAD: Assessment Data

Module	Table	Records	What it contains (main fields)
VLE interactions	interactions	10,655,280	Clickstream data: `code_module`, `code_presentation`, `id_student`, `id_site`, `date`, `sum_click`
VLE interactions	vle	6,364	VLE resource metadata: `id_site`, `code_module`, `code_presentation`, `activity_type`, `week_from`, `week_to`

Table 3: OULAD: Virtual Learning Environment Interactions

To correctly and effectively classify the students' final results, preprocessing must be done to merge the tables and create a single table containing all the features, keyed according to the `id_student` column. In the next subsection, we describe the preprocessing steps taken to achieve this.

We divide the tables into three groups: background (studentInfo), performance (studentAssessment and assessments), and behavior (studentVle, vle, and assessments for cutoff construction). We thus perform the following preprocessing tasks on each group as follows:

Background: We first perform one-hot encoding on the categorical features in the `studentInfo` table, which include students' gender, region, highest education and disability status. For age and IMD bands, we perform ordinal encoding, as these features have an inherent order.

Performance: We first merge the table `studentAssessment` and the table `assessments` on `id_assessment`. Missing due dates and scores are explicitly tracked using binary indicators (`has_due_date` and `score_missing`), and missing values are imputed using the median. We then engineer timing features from submission behavior, including `submission_delay`, late/early flags, and a capped delay variable based on the 1st and 99th percentiles to reduce outlier effects. In addition, we compute weighted performance (`weighted_score`) using assessment weights and its min-max normalized version. Finally, we aggregate all assessment records at the student-module-presentation level (`id_student`, `code_module`, `code_presentation`) to produce summary statistics (counts, central tendency, variability, weighted-score totals, lateness ratios, delay average, and missingness ratios), and append assessment-type frequency features via a pivot table.

Behavior: To construct behavior features from VLE interactions, we first derive a module-presentation cutoff date as the latest assessment date and keep only clicks that occur on or before this cutoff, preventing post-assessment leakage. We then merge `studentVle` with `vle` to attach `activity_type` for each `id_site` (with missing types labeled as unknown). Next, we aggregate `sum_click` values per student-module-presentation and activity type, and pivot them into

wide features (e.g., `clicks_forum`, `clicks_resource`). Because interactions are processed in chunks for scalability, we finally sum partial chunk-level results to obtain one behavior-feature row per (`id_student`, `code_module`, `code_presentation`).

After preprocessing, we aggregate three groups of features, using `student_id` as the key, to create the final dataset for modeling. This dataset contains 32,593 records and 72 features, including the students' ID and their final results. The final results are categorized into four classes: `Distinction`, `Pass`, `Fail`, and `Withdrawn`.

Next, we split the dataset into training and testing sets, with an 80-20 split. We use `sk_learn`'s `SMOTE` to address class imbalance in the training set, which is a common issue in educational datasets where certain outcomes (e.g., withdrawal) may be more prevalent than others. This increases the size of the train set to 39,556 records, which raises the total dataset size to 46,076 records.

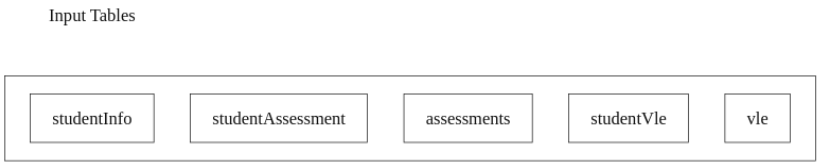

Fig. 1: Preprocessing pipeline (Stage 1): input tables.

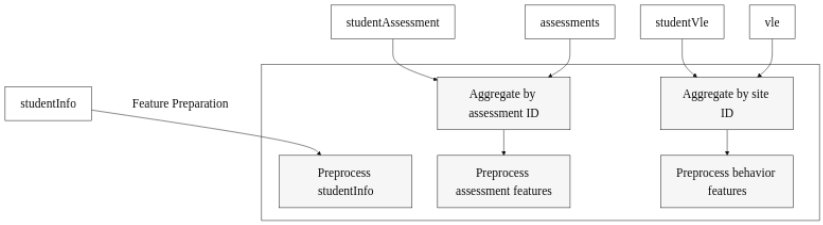

Fig. 2: Preprocessing pipeline (Stage 2): feature preparation.

4.2 Modeling

Traditional Machine Learning Models In our paper, we utilize seven traditional machine learning models as baselines for comparison with large language models. These models include: Random Forest (RF), Support Vector Machine (SVM), Gradient Boosting

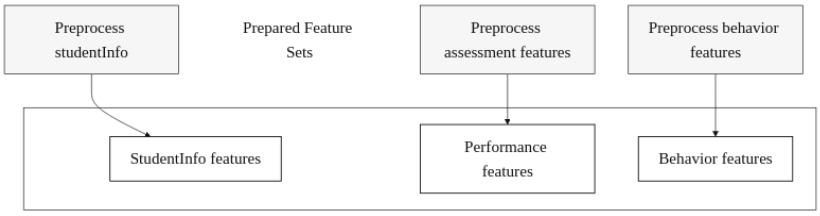

Fig. 3: Preprocessing pipeline (Stage 3): prepared feature sets.

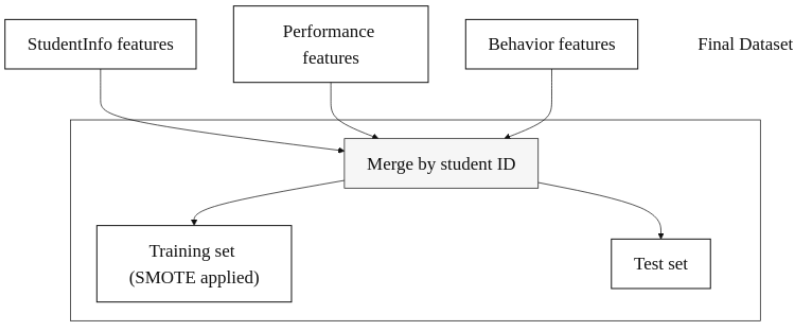

Fig. 4: Preprocessing pipeline (Stage 4): final dataset construction.

(GB), Decision Tree (DT), K-Nearest Neighbors (KNN), Extreme Learning Machine (ELM), and Multi-layer Perceptron (MLP).

The first two models, RF and SVM, are learning methods identified as top classifiers in Delgado et al.'s systematic review of educational data mining [14]. The paper also highlights other strong models, such as DT (C5.0 implementation), GB, ELM, and MLP. A review of the OULAD dataset by Alhakbani and Alnassar [1] also identifies DT (CART implementation) as a top-performing model in detecting at-risk students. Finally, KNN is also included, due to its simplicity, as we intend to explore if the classification task can be effectively solved using a simple distance-based method.

Regarding implementation, all models are run with `sk_learn` libraries; for the DT model, we also uses R programming language to implement the C5.0 algorithm, since `sk_learn` only provides the CART implementation. We first run a 5-fold cross-validation on the training set to perform hyperparameter tuning for each model. We then select the best hyperparameters based on the average F1-score across folds, and train the final model on the entire training set using these hyperparameters.

Large Language Models For baseline LLMs, we utilize two models available on Huggin Face: `Qwen-2.5-3b` and `LlaMa-3-8b`. The models are not fine-tuned and was prompted to perform the classification task with zero-shot prompting.

Model	**Hyperparameters**
Random Forest	`n_estimators=300, min_samples_split=5, min_samples_leaf=2, max_features='sqrt', max_depth=50, bootstrap=False`
Decision Tree	`splitter='best', min_samples_split=20, min_samples_leaf=4, max_features=None, max_depth=10, criterion='gini'`
Support Vector Machine	`StandardScaler() + SVC(C=0.7847599703514611, class_weight='balanced', kernel='rbf', gamma='auto')`
Gradient Boosting	`subsample=1.0, n_estimators=100, min_samples_split=2, min_samples_leaf=4, max_features=None, max_depth=5, learning_rate=0.2`
K-NN	`StandardScaler() + KNeighborsClassifier(n_neighbors=7, weights='uniform', p=1, metric='minkowski', leaf_size=50)`
Extreme Learning Machine	`ELMClassifier(n_neurons=1000, alpha=0.0001, ufunc='relu')`
Multi-layer Perceptron	`MLPClassifier(hidden_layer_sizes=(32,), activation='tanh', alpha=0.1, solver='lbfgs', max_iter=1500, max_fun=15000)`

Table 4: Hyperparameters used for each traditional machine learning model.

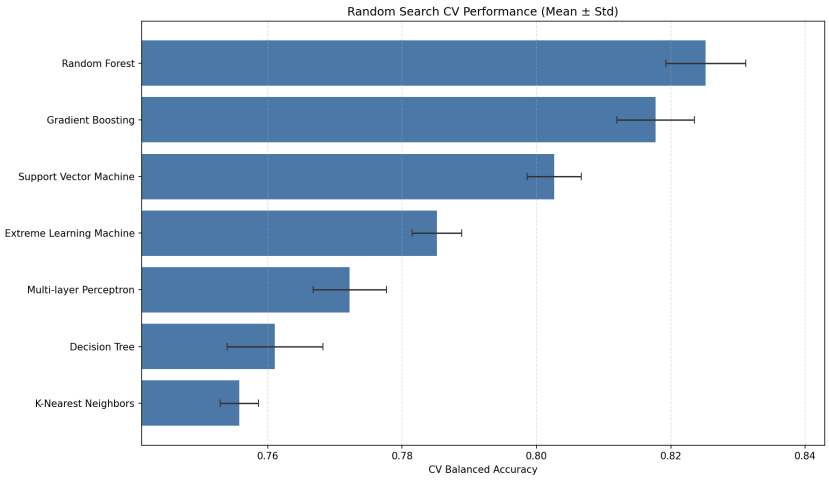

Fig. 5: Cross-validation results for traditional machine learning models.

Listing 1.1: Function to build LLMs prompt

```
def build_prompt(
    base_prompt: str,
    row_dict: dict,
    few_shot_examples: list[tuple[dict, str]],
) -> str:
    current_row_json = json.dumps(row_dict, ensure_ascii=False)

    parts = [
        base_prompt,
        "Allowed labels: Distinction, Pass, Withdrawn, Fail.",
        "Infer the label from the pattern in the examples.",
        "Do not default to the most common example label.",
        "Output exactly one line in this format:",
        "LABEL=<one of Distinction, Pass, Withdrawn, Fail>",
        "Do not output anything else.",
    ]

    if few_shot_examples:
        parts.append("")
        parts.append("Examples:")

        for idx, (example_x, example_y) in enumerate(few_shot_examples,
    start=1):
            example_x_json = json.dumps(sanitize_row(example_x),
    ensure_ascii=False)
            parts.extend(
                [
                    f"Example {idx}:",
                    f"Student record:\n{example_x_json}",
                    f"LABEL={example_y}",
                    "",
                ]
            )

    parts.extend(
        [
            "Now classify this record:",
            f"Student record:\n{current_row_json}",
            "Answer:",
        ]
    )

    return "\n".join(parts)
```

5 Results

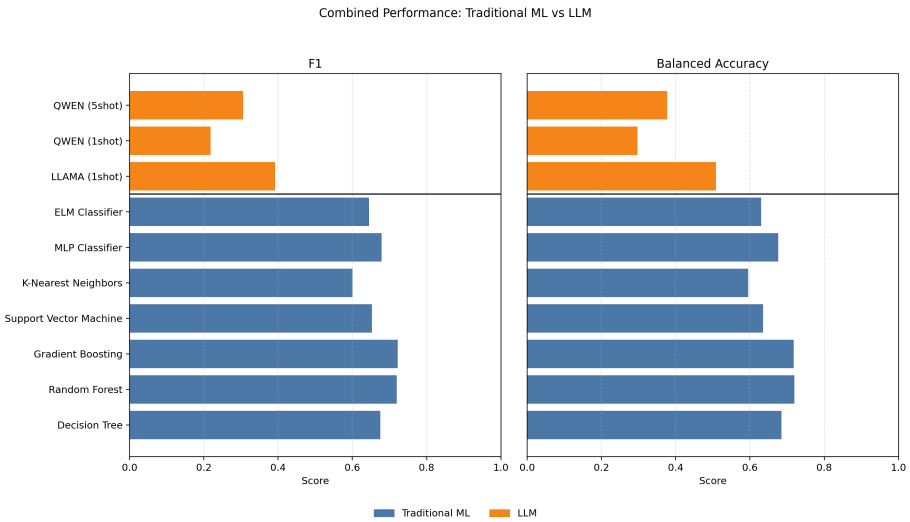

Fig. 6: Combined performance comparison of traditional machine learning models and large language models using F1 and Balanced Accuracy.

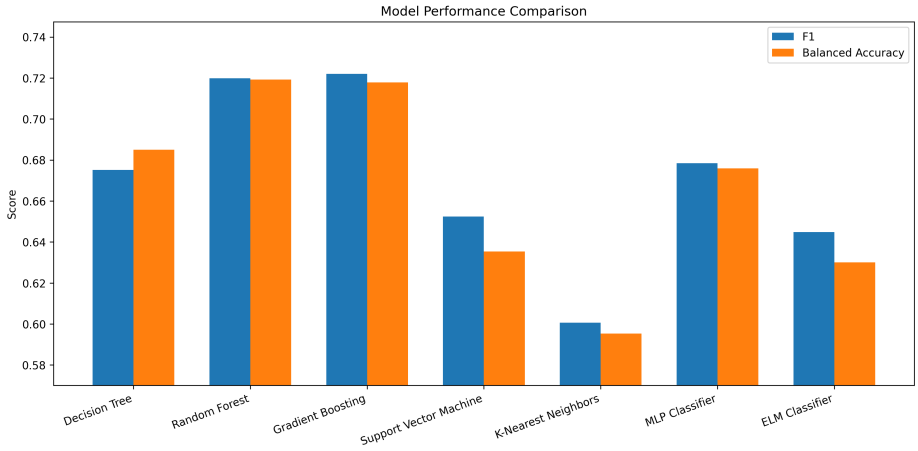

Fig. 7: Test-set performance comparison of traditional predictive models using Macro F1 and Balanced Accuracy.

5.1 Traditional Predictive Models

Various traditional model architectures are implemented. The evaluated models are Random Forest, Decision Tree, Support Vector Machine, Gradient Boosting, K-Nearest Neighbors, Extreme Learning Machine, and Multilayer Perceptron. The performance of each model is measured using plain accuracy, balanced accuracy, and F1 score. The accuracy of a model can be calculated as

$$\frac{\text{True Positive} + \text{True Negative}}{\text{Total Prediction}}.$$

It is an intuitive measure of how correct the model predictions are, but falls short when the given dataset is imbalanced. To mitigate this issue, the balanced accuracy is preferred:

$$\frac{\text{Sensitivity} + \text{Specificity}}{2}.$$

In addition, the F1 score is also utilized, calculated as the harmonic mean of precision and recall:

$$\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

As shown in Fig. 1, the model performance plot summarizes the comparative results across all evaluated traditional predictive models.

5.2 Large Language Models

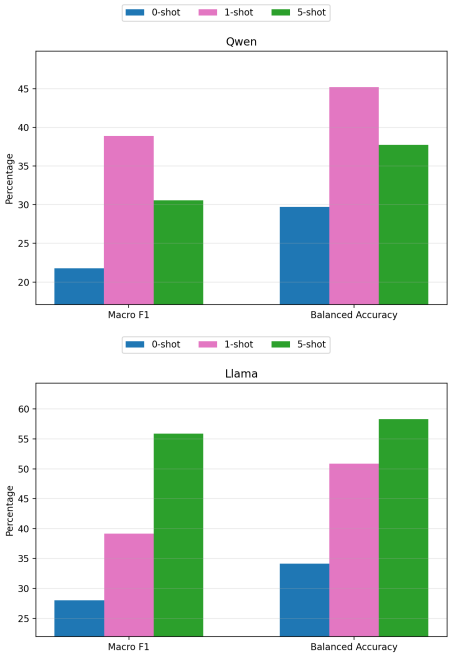

Fig. 8: Prompting-technique comparison for Qwen2.5-3B (left) and Meta-Llama3-9B (right), using Macro F1 and Balanced Accuracy.

We use a modified metric for LLMs—Macro F1. This metric effectively treats each class in a model equally important by taking the mean of the F1 scores of each class. Observe that it performs noticeably worse than the lowest scoring traditional predictive model.

6 Discussion

6.1 Interpretation

The results highlight immense accuracy of traditional machine learning

models, specifically Gradient Boosting. In particular, the machine learning model Gradient Boosting predict the students' final result with a balanced accuracy of just under 0.8. This suggests that students' access to study materials, as well as study time, greatly affect the students' academic trajectory.

On the other hand, LLMs show a consistently poor performance, achieving a balanced accuracy of no further than .5. This is due to the fact that LLMs tend to witness a phenomenon of fallthrough to a default heuristics, most commonly "Fail" or "Pass". Interestingly, one-shot prompts perform better by a significant margin at, but five-shot prompts perform no better than zero-shot.

From a pedagogical standpoint, our study serves as a rudimentary framework for implementating an early warning system for students. This cements the role of traditional machine learning models for such purpose and illustrates the unreliability of generative LLMs in classification problems.

6.2 Threats to Validity

Both models feed upon the OULAD, which lacks access to external factors that may tamper with students' academic performance. This includes but are not limited to student mental health and sudden students' emergencies, acting as black swans to throw off models' accuracies.

There exists a risk of models overfitting to data in more obsolete academic backgrounds, which reinforces their biases to the more recent education system. Furthermore, the data yielded from the students who are aware that their activities are monitored may reflect a different study pattern than those who are not, which drastically change the models' prediction accuracy.

7 Conclusion

This study presented a thorough analysis between the performances of traditional machine learning models and Large Language Models with the view of predicting students' performances based on the students' footprint on the online learning system and active hours, interpreted as the students' study patterns. The result found a remarkably better performance of the former than the latter, reinforcing the effectiveness of Random Forests and Gradient Boosting—the primary vessels for classification problems.

Our future work aims to refine the prompts fed to the LLMs for more accurate predictions, as well as analyses of more technically demanding LLMs compared to those in this study. Furthermore, additional socioeconomic data of the students, as well as a more diverse dataset across disciplines, would aid in the generalizability analysis of the engagement features.

References

1. Alhakbani, H.A., Alnassar, F.M.: Open Learning Analytics: A Systematic Review of Benchmark Studies using Open University Learning Analytics Dataset (OULAD). In: 2022 7th International Conference on Machine Learning Technolo-

gies (ICMLT). ICMLT 2022, pp. 81–86. ACM (2022). https://doi.org/10.1145/3529399.3529413
2. Alias, M.A.H., Abdul Aziz, M.A., Hambali, N., Taib, M.N.: Student performance classification: a comparison of feature selection methods based on online learning activities. International Journal of Electrical and Computer Engineering (IJECE) **14**(4), 4675 (2024). https://doi.org/10.11591/ijece.v14i4.pp4675-4685
3. Andretta Jaskowiak, P., Campello, R.: Comparing Correlation Coefficients as Dissimilarity Measures for Cancer Classification in Gene Expression Data. In: (2011)
4. Breiman, L.: Random Forests. Machine Learning **45**(1), 5–32 (2001). https://doi.org/10.1023/a:1010933404324
5. Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D.: Language Models are Few-Shot Learners, (2020). https://doi.org/10.48550/ARXIV.2005.14165.
6. Brzezinski, D., Stefanowski, J.: Prequential AUC: properties of the area under the ROC curve for data streams with concept drift. Knowledge and Information Systems **52**(2), 531–562 (2017). https://doi.org/10.1007/s10115-017-1022-8
7. Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: SMOTE: Synthetic Minority Over-sampling Technique. (2011). https://doi.org/10.48550/ARXIV.1106.1813
8. Chen, H.-H.: Understanding Gradient Boosting Classifier: Training, Prediction, and the Role of γ_j. (2024). https://doi.org/10.48550/ARXIV.2410.05623. arXiv: 2410.05623 [cs.LG]
9. Cortes, C., Vapnik, V.: Support-vector networks. Machine Learning **20**(3), 273–297 (1995). https://doi.org/10.1007/bf00994018
10. Cover, T., Hart, P.: Nearest neighbor pattern classification. IEEE Transactions on Information Theory **13**(1), 21–27 (1967). https://doi.org/10.1109/tit.1967.1053964
11. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, (2018). https://doi.org/10.48550/ARXIV.1810.04805.
12. Elreedy, D., Atiya, A.F.: A Comprehensive Analysis of Synthetic Minority Oversampling Technique (SMOTE) for handling class imbalance. Information Sciences **505**, 32–64 (2019). https://doi.org/10.1016/j.ins.2019.07.070
13. Everitt, B.S., Landau, S., Leese, M., Stahl, D.: Cluster Analysis. Wiley (2011)
14. Fernández-Delgado, M., Cernadas, E., Barro, S., Amorim, D.: Do we Need Hundreds of Classifiers to Solve Real World Classification Problems? Journal of Machine Learning Research **15**(90), 3133–3181 (2014). http://jmlr.org/papers/v15/delgado14a.html
15. Friedman, J.H.: Greedy function approximation: A gradient boosting machine. The Annals of Statistics **29**(5) (2001). https://doi.org/10.1214/aos/1013203451
16. Haykin, S.S.: Neural networks. A comprehensive foundation. Prentice Hall, Upper Saddle River, NJ (1999)
17. Ho, T.K.: Random decision forests. In: Proceedings of 3rd International Conference on Document Analysis and Recognition. ICDAR-95, pp. 278–282. IEEE Comput. Soc. Press. https://doi.org/10.1109/icdar.1995.598994
18. Hsu, C.-w., Chang, C.-c., Lin, C.-J.: A Practical Guide to Support Vector Classification Chih-Wei Hsu, Chih-Chung Chang, and Chih-Jen Lin. (2003)

19. Huang, G.-B., Zhou, H., Ding, X., Zhang, R.: Extreme Learning Machine for Regression and Multiclass Classification. IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics) **42**(2), 513–529 (2012). https://doi.org/10.1109/tsmcb.2011.2168604
20. Huang, G.-B., Zhu, Q.-Y., Siew, C.-K.: Extreme learning machine: Theory and applications. Neurocomputing **70**(1–3), 489–501 (2006). https://doi.org/10.1016/j.neucom.2005.12.126
21. Keathley-Herring, H., Van Aken, E., Gonzalez-Aleu, F., Deschamps, F., Letens, G., Orlandini, P.C.: Assessing the maturity of a research area: bibliometric review and proposed framework. Scientometrics **109**(2), 927–951 (2016). https://doi.org/10.1007/s11192-016-2096-x
22. Kuzilek, J., Hlosta, M., Zdrahal, Z.: Open University Learning Analytics dataset. Scientific Data **4**(1) (2017). https://doi.org/10.1038/sdata.2017.171
23. Nigsch, F., Bender, A., van Buuren, B., Tissen, J., Nigsch, E., Mitchell, J.B.O.: Melting Point Prediction Employing k-Nearest Neighbor Algorithms and Genetic Parameter Optimization. Journal of Chemical Information and Modeling **46**(6), 2412–2422 (2006). https://doi.org/10.1021/ci060149f
24. Rosenblatt, F.: The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review **65**(6), 386–408 (1958). https://doi.org/10.1037/h0042519
25. Schölkopf, B., Smola, A.J.: Learning with Kernels: support vector machines, regularization, optimization, and beyond. MIT Press (2002)
26. Suleiman, I.B., Okunade, O.A., Dada, E.G., Ezeanya, U.C.: Key factors influencing students' academic performance. Journal of Electrical Systems and Information Technology **11**(1) (2024). https://doi.org/10.1186/s43067-024-00166-w
27. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention Is All You Need, (2017). https://doi.org/10.48550/ARXIV.1706.03762.
28. Yang, Z., Zhang, T., Lu, J., Zhang, D., Kalui, D.: Optimizing area under the ROC curve via extreme learning machines. Knowledge-Based Systems **130**, 74–89 (2017). https://doi.org/10.1016/j.knosys.2017.05.013
29. Zaky, U., Naswin, A., Sumiyatun, S., Murdiyanto, A.W.: Performance Analysis of the Decision Tree Classification Algorithm on the Water Quality and Potability Dataset. Indonesian Journal of Data and Science **4**(3), 145–150 (2023). https://doi.org/10.56705/ijodas.v4i3.113